

ELL7283 — Embedded Systems (PG)

## Assignment 2

---

# Network-Synchronised Speaking Clock

---

*using STM32, FreeRTOS, and QEMU*

*Submitted by*

**Aditya Jha**

2025EET2479

**Aman Mongre**

2025EET2488

**Indian Institute of Technology Delhi**

April 2026

# 1. System Architecture

The speaking clock runs entirely inside the QEMU emulator — no physical development board is required. The system is structured as four layers.

| Layer      | Component              | Role                                                |
|------------|------------------------|-----------------------------------------------------|
| Firmware   | FreeRTOS V11.1.0 tasks | Keypress detection, NTP fetch, token generation     |
| Networking | lwIP 2.x               | UDP/IP/DHCP/DNS; netconn API for NTP client         |
| Emulation  | QEMU MPS2-AN500        | Cortex-M7 CPU, LAN9118 NIC, UART0 console           |
| Host       | tts_bridge.py          | Collects TOKEN lines, assembles sentence, speaks it |

## 1.1 Source Directory Structure

All project-specific code is confined to `src/`, `network/`, `include/`, `config/`, `linker/`, and `startup/`. The `freertos/` and `lwip/` directories contain the upstream library source trees required for compilation.

```
speaking_clock/
├── src/          main.c  key_task.c  ntp_task.c  speech_task.c  syscalls.c
├── network/      lwip_init.c  ntp_client.c  netif_qemu.c  sys_arch.c  cc.h
├── include/      speech_tokens.h  time_msg.h
├── config/       FreeRTOSConfig.h  lwipopts.h
├── linker/       stm32.ld
├── startup/      startup_stm32.s
├── freertos/     (FreeRTOS kernel - required for compilation)
├── lwip/         (lwIP 2.x source - required for compilation)
├── tts_bridge.py
└── Makefile
```

Two additional files beyond the skeleton are present and are required. `syscalls.c` provides the newlib `_write` stub that routes `printf` to ARM semihosting so that firmware output reaches QEMU's console. `sys_arch.c` implements the lwIP FreeRTOS OS abstraction layer (mutexes, semaphores, and thread creation) required by lwIP's `NO_SYS = 0` mode.

## 1.2 Inter-Task Communication

The three FreeRTOS tasks exchange data through two binary semaphores and one message queue. A third semaphore synchronises the NTP task with DHCP completion.

| From          | To          | IPC Object                          | Purpose                                           |
|---------------|-------------|-------------------------------------|---------------------------------------------------|
| Key Task      | NTP Task    | time_request_sem (binary semaphore) | Signals that the user has pressed 't'             |
| DHCP callback | NTP Task    | dhcp_ready_sem (binary semaphore)   | Signals that a valid IP address has been assigned |
| NTP Task      | Speech Task | time_queue (queue, depth 1)         | Passes hour, minute, second in IST                |

| From        | To   | IPC Object                 | Purpose                                   |
|-------------|------|----------------------------|-------------------------------------------|
| Speech Task | Host | UART0 / semihosting stdout | TOKEN and END lines read by tts_bridge.py |

## 2. RTOS Design

The firmware uses FreeRTOS V11.1.0 configured with a preemptive scheduler. Three application tasks are created in main.c before vTaskStartScheduler() is called. The lwIP stack creates two additional internal tasks — netif\_rx and tcpip\_thread — during network initialisation.

### 2.1 FreeRTOS Configuration

| Parameter                     | Value    | Purpose                                            |
|-------------------------------|----------|----------------------------------------------------|
| configUSE_PREEMPTION          | 1        | Preemptive scheduling                              |
| configCPU_CLOCK_HZ            | 25000000 | 25 MHz system clock for MPS2-AN500                 |
| configTICK_RATE_HZ            | 1000     | 1 ms tick resolution                               |
| configMAX_PRIORITIES          | 8        | Supports application tasks and lwIP internal tasks |
| configTOTAL_HEAP_SIZE         | 131072   | 128 KB heap; heap_4.c first-fit allocator          |
| configUSE_MUTEXES             | 1        | Required by lwIP sys_arch layer                    |
| configUSE_RECURSIVE_MUTEXES   | 1        | Required by lwIP sys_arch layer                    |
| configUSE_COUNTING_SEMAPHORES | 1        | Required by lwIP                                   |

### 2.2 Task Summary

| Task                | Stack (words) | Priority | Responsibility                                                                  |
|---------------------|---------------|----------|---------------------------------------------------------------------------------|
| key_task            | 256           | 1        | Polls UART0 every 100 ms; gives time_request_sem on 't' keypress                |
| ntp_task            | 1024          | 2        | Waits for DHCP, then for keypress; fetches NTP time; sends result to time_queue |
| speech_task         | 512           | 1        | Receives time_msg; prints TOKEN lines and END to stdout                         |
| netif_rx (lwIP)     | —             | 4        | Polls LAN9118 RX FIFO for incoming Ethernet frames                              |
| tcpip_thread (lwIP) | —             | 5        | Runs all lwIP protocol processing                                               |

## 2.3 Key Input Task

key\_task reads the CMSDK APB UART0 status register at 0x40004000 directly instead of using the blocking semihosting getchar(). This way the task doesn't hold the CPU while waiting — it checks every 100 ms and yields between checks. Both 't' and 'T' are accepted.

```
int c = uart_getchar_nb();
if (c == 't' || c == 'T')
    xSemaphoreGive(time_request_sem);
vTaskDelay(pdMS_TO_TICKS(100));
```

## 2.4 NTP Client Task

When ntp\_task starts it waits on dhcp\_ready\_sem with a 30-second timeout. The lwIP netif status callback in lwip\_init.c gives this semaphore once DHCP assigns an IP, so there is no polling loop needed. After that the task just blocks on time\_request\_sem waiting for a keypress. Each time it wakes up it calls ntp\_get\_time() and puts the result on time\_queue.

```
xSemaphoreTake(dhcp_ready_sem, pdMS_TO_TICKS(30000));
while (1) {
    xSemaphoreTake(time_request_sem, portMAX_DELAY);
    time_msg msg;
    if (ntp_get_time(&msg))
        xQueueSend(time_queue, &msg, pdMS_TO_TICKS(1000));
}
```

## 2.5 Speech Task and Token Output

speech\_task blocks on xQueueReceive(time\_queue, &msg, portMAX\_DELAY). When a time\_msg arrives it calls emit\_time\_tokens(), which converts the hour, minute, and second integers to English words and prints them as TOKEN lines followed by END.

Number-to-word conversion uses two lookup arrays: ones[] for values 0–19 and tens\_words[] for 20–50. Compound numbers above 19 are emitted as two tokens joined by a space (e.g. TWENTY TWO). Hours are validated against 0–23 and minutes/seconds against 0–59. All token string literals are defined as preprocessor constants in speech\_tokens.h so there are no scattered bare strings in the source.

The firmware does not generate audio directly. tts\_bridge.py reads the TOKEN lines from QEMU's stdout, joins them into a sentence in natural sentence case, and passes it to the system TTS engine (say on macOS, espeak-ng on Linux, PowerShell SpeechSynthesizer on Windows).

```
TOKEN THE
TOKEN TIME
TOKEN IS
TOKEN TWENTY TWO
TOKEN HOURS
TOKEN TWENTY FIVE
TOKEN MINUTES
TOKEN AND
TOKEN THIRTY FOUR
```

```
TOKEN SECONDS
END
```

tts\_bridge.py is launched automatically by the make qemu-tts Makefile target via a shell pipe:

```
qemu-system-arm ... -nographic -semihosting ... 2>&1 | python3 tts_bridge.py
```

## 2.6 IPC Initialisation Order in main.c

All three IPC objects are created before any tasks are started. dhcp\_ready\_sem must be created before network\_init() is called because the DHCP callback registered inside lwip\_init.c requires the semaphore to exist before tcpip\_thread starts.

```
dhcp_ready_sem    = xSemaphoreCreateBinary();    /* before network_init() */
network_init();   /* creates tcpip_thread */
time_request_sem  = xSemaphoreCreateBinary();
time_queue        = xQueueCreate(1, sizeof(time_msg));
```

## 3. Networking Implementation

---

Networking uses lwIP 2.x in FreeRTOS mode (NO\_SYS = 0), where lwIP runs its own tcpip\_thread task and all protocol processing is done inside that task. Only the protocols required by the assignment are enabled: UDP, IP, DHCP, and DNS. TCP is explicitly disabled in lwipopts.h to reduce memory footprint. The NTP client uses the lwIP netconn API, which provides a simpler blocking interface compared to the raw callback-based PCB API.

### 3.1 lwIP Configuration (lwipopts.h)

| Option         | Value | Reason                                           |
|----------------|-------|--------------------------------------------------|
| NO_SYS         | 0     | FreeRTOS integration — lwIP runs in tcpip_thread |
| LWIP_UDP       | 1     | Required for NTP (port 123)                      |
| LWIP_TCP       | 0     | Not required; disabled to reduce RAM usage       |
| LWIP_DHCP      | 1     | Obtains IP address from QEMU SLIRP               |
| LWIP_DNS       | 1     | Resolves NTP server hostnames                    |
| LWIP_NETCONN   | 1     | Enables netconn API used in ntp_client.c         |
| MEM_SIZE       | 16384 | 16 KB lwIP heap                                  |
| PBUF_POOL_SIZE | 8     | Packet buffer pool depth                         |

### 3.2 Virtual Ethernet Driver (netif\_qemu.c)

The LAN9118 Ethernet controller is emulated by QEMU at base address 0xA0000000 on the MPS2-AN500 platform. The driver performs a full hardware reset via HW\_CFG\_SRST, verifies the chip identity by reading the ID\_REV register (expected value: 0x01180001), programs the MAC address to 52:54:00:12:34:56, and enables TX and RX via the MAC control register. A dedicated netif\_rx task at priority 4 continuously polls the RX FIFO for incoming Ethernet frames.

### 3.3 DHCP Startup Sequence

The order of calls in `lwip_init.c` matters. `tcpip_init()` creates `tcpip_thread` first. `netif_set_status_callback()` is registered before `dhcp_start()` so the callback is already in place when the DHCP reply arrives. When DHCP assigns an IP the callback calls `xSemaphoreGive(dhcp_ready_sem)` which wakes `ntp_task`. In practice QEMU SLIRP assigns 10.0.2.15 (gateway 10.0.2.2, DNS 10.0.2.3) within about 1–2 seconds of the scheduler starting.

### 3.4 DNS Resolution and NTP UDP Socket

The NTP client calls `netconn_gethostbyname()` to resolve the server hostname. Six servers are tried in order: 0.in.pool.ntp.org, 1.in.pool.ntp.org, 2.in.pool.ntp.org, time.nplindia.org, time.cloudflare.com, and time.google.com as a last resort. The Indian pool servers are listed first since the assignment recommends them. The global anycast servers (Cloudflare, Google) are kept as fallbacks because they tend to be more reliably reachable from QEMU SLIRP, where all outgoing traffic shares the host's public IP and the pool servers can throttle requests if hit too frequently. For each server the client opens a new UDP netconn, left unconnected, and binds it to a local port in the range 50000–59999 (incremented each call). The PCB is kept unconnected because anycast servers like Cloudflare or Google may reply from a different IP than the one the packet was sent to — a connected PCB would drop those replies silently. An explicit local port is used so QEMU SLIRP opens a fresh host socket each time, avoiding stale flow entries that can cause outgoing packets to be discarded on repeat calls. The request goes out via `netconn_sendto()` and the reply is read with `netconn_recv()` with a 5-second timeout. A minimum gap of 15 seconds (`NTP_MIN_INTERVAL_MS`) is enforced between NTP requests using FreeRTOS tick counts. If 't' is pressed again before that time is up, `ntp_get_time()` waits out the remainder before sending the next request — this avoids getting rate-limited by the pool servers. The mode field in the response (byte 0, bits 2:0) is checked to be 4 (server mode); anything else gets logged and skipped.

## 4. NTP Protocol Explanation

The Network Time Protocol operates over UDP on port 123 and is specified in RFC 5905. A client sends a 48-byte request and extracts the server's current time from the 48-byte response. No authentication or symmetric-mode handshaking is required for a basic client. NTPv4 is used (Version Number = 4) as it is the current standard accepted by all public NTP servers used in this assignment.

### 4.1 NTP Packet Structure

| Byte Offset | Field                        | Width   | Notes                                         |
|-------------|------------------------------|---------|-----------------------------------------------|
| 0           | LI, VN, Mode                 | 1 byte  | Set to 0x23 in request                        |
| 1           | Stratum                      | 1 byte  | Zero in request; filled by server in response |
| 2–3         | Poll / Precision             | 2 bytes | Zero in request                               |
| 4–11        | Root Delay / Root Dispersion | 8 bytes | Zero in request                               |

| Byte Offset | Field                                   | Width    | Notes                                                          |
|-------------|-----------------------------------------|----------|----------------------------------------------------------------|
| 12–15       | Reference ID                            | 4 bytes  | Zero in request                                                |
| 16–39       | Reference / Origin / Receive Timestamps | 24 bytes | Zero in request                                                |
| 40–43       | Transmit Timestamp (seconds)            | 4 bytes  | Extracted from response — seconds since 1 Jan 1900, big-endian |
| 44–47       | Transmit Timestamp (fraction)           | 4 bytes  | Ignored — 1-second resolution is sufficient                    |

## 4.2 Request Packet Construction

Only the first byte of the request packet needs to be set; the remaining 47 bytes are zero. The bit layout of byte 0 is: LI = 00 (no leap second warning), VN = 100 (NTPv4), Mode = 011 (client), giving the value 0x23.

```
uint8_t pkt[48];
memset(pkt, 0, sizeof(pkt));
pkt[0] = 0x23; /* LI=0 VN=4 Mode=3 (client) */
```

## 4.3 Timestamp Extraction and IST Conversion

The transmit timestamp at bytes 40–43 is a big-endian 32-bit count of seconds since 1 January 1900 (the NTP epoch). The four bytes are assembled manually to avoid any dependency on host endianness. Two offsets are then applied to obtain Indian Standard Time.

First, 2,208,988,800 is subtracted to convert from the NTP epoch to the Unix epoch — this figure represents the exact number of seconds between 1 January 1900 and 1 January 1970, accounting for all 17 leap years in that interval. Second, 19,800 seconds are added for the IST offset of UTC+5:30 ( $5 \times 3600 + 30 \times 60 = 19,800$ ). A validity guard rejects any server timestamp smaller than the NTP-to-Unix offset, which would indicate a zero or malformed response.

```
uint32_t secs = ((uint32_t)data[40] << 24) | ((uint32_t)data[41] << 16)
               | ((uint32_t)data[42] << 8) | (uint32_t)data[43];
if (secs < 2208988800UL) return 0; /* reject invalid timestamp */
uint32_t unix_time = secs - 2208988800UL; /* NTP epoch -> Unix epoch */
uint32_t ist_time = unix_time + 19800UL; /* UTC -> IST (+5:30) */
msg->hour = (ist_time / 3600) % 24;
msg->minute = (ist_time / 60) % 60;
msg->second = ist_time % 60;
```

## 5. System Testing

The complete assignment was tested end-to-end using the `make qemu-tts` Makefile target. Testing was performed at multiple wall-clock times to verify IST conversion accuracy, correct token generation, and speech output.

### 5.1 Build and Execution

The assignment is compiled and run with a single command:

```
make qemu-tts
```

This invokes `arm-none-eabi-gcc` with the following flags, links the ELF image against the FreeRTOS and lwIP libraries, and launches QEMU piped into `tts_bridge.py`:

```
-mcpu=cortex-m7 -mthumb -mfpv5-d16 -mfloat-abi=hard -O0 -g3 -Wall -Wextra  
-Wno-unused-parameter  
qemu-system-arm -machine mps2-an500 -cpu cortex-m7 \  
-kernel speaking_clock.elf -nographic -semihosting \  
-net nic,model=lan9118 -net user 2>&1 | python3 tts_bridge.py
```

The linker script (`linker/stm32.ld`) places the vector table and code in FLASH at `0x00000000`, and stack/heap in RAM at `0x20000000`, consistent with the MPS2-AN500 memory map.

### 5.2 Speech Output Verification

The following IST times were verified against the required spoken format:

| IST Time | Spoken Output                                                            |
|----------|--------------------------------------------------------------------------|
| 22:25:34 | The time is twenty two hours twenty five minutes and thirty four seconds |
| 14:32:05 | The time is fourteen hours thirty two minutes and five seconds           |
| 00:00:00 | The time is zero hours zero minutes and zero seconds                     |
| 09:05:01 | The time is nine hours five minutes and one seconds                      |
| 23:59:59 | The time is twenty three hours fifty nine minutes and fifty nine seconds |

### 5.3 Test Procedure

| Step | Action                         | Expected Observation                                                                | Result |
|------|--------------------------------|-------------------------------------------------------------------------------------|--------|
| 1    | Run <code>make qemu-tts</code> | Firmware compiles with zero warnings; QEMU starts; TTS bridge begins watching stdin | PASS   |
| 2    | Boot messages                  | [MAIN] FreeRTOS V11.1.0+ and [NIC] LAN9118 ID_REV = 0x01180001 appear               | PASS   |

| Step | Action                           | Expected Observation                                                 | Result |
|------|----------------------------------|----------------------------------------------------------------------|--------|
| 3    | Wait for DHCP                    | [LWIP] DHCP assigned: 10.0.2.15 appears within ~2 seconds            | PASS   |
| 4    | Press 't'                        | [KEY] Time request received; NTP task wakes                          | PASS   |
| 5    | DNS resolution                   | [NTP_CLIENT] Trying 0.in.pool.ntp.org; [NTP_CLIENT] Resolved -> <IP> | PASS   |
| 6    | NTP fetch and IST conversion     | [NTP] Time: HH:MM:SS IST — value matches host system clock           | PASS   |
| 7    | TOKEN stream                     | TOKEN THE ... TOKEN SECONDS followed by END printed                  | PASS   |
| 8    | Speech output                    | [BRIDGE] Speaking: ... printed; TTS engine announces the time        | PASS   |
| 9    | Repeat keypress                  | System re-fetches a fresh NTP time without restarting QEMU           | PASS   |
| 10   | 'T' (capital) accepted           | Same behaviour as lowercase 't'                                      | PASS   |
| 11   | Midnight boundary (23:59:59 IST) | hour=23, minute=59, second=59; modulo-24 wrap correct                | PASS   |

## 5.4 Terminal Output

The screenshot below captures the full QEMU terminal output from a representative test run. It covers all four demonstration requirements stated in the assignment: QEMU running, user keypress, network time retrieval, and spoken time output.

```

aditya@Adityas-MacBook-Air:~$ make qemu-tts
text data bss dec hex filename
68252 252 195428 263932 486fc speaking_clock.elf
qemu-system-arm -machine mps2-an500 -cpu cortex-m7 -kernel speaking_clock.elf -nographic -semihosting -net nic,model=lan9118 -net user 2>&1 | python3 tts_bridge.py

Speaking Clock

Watching for TOKEN/END stream from QEMU firmware...
Press Ctrl+C to stop.

FIRMWARE BOOT

=== Speaking Clock (QEMU mps2-an500) ===
[MAIN] FreeRTOS V11.1.0+
[NIC] LAN9118 ID_REV = 0x01180001
[NIC] LAN9118 ready, MAC 52:54:00:12:34:56

--- Network Init ---
[LWIP] DHCP started - waiting for address assignment
[MAIN] Starting scheduler
[NTP] Waiting for DHCP...
[KEY] Press 't' to speak current IST time
[SPEECH] Task started - waiting for time

--- NTP Request ---
[KEY] Time request received
[LWIP] DHCP assigned: 10.0.2.15
[NTP] Network ready: 10.0.2.15
[NTP] Press 't' to get current IST time
[NTP] Fetching time...
[NTP_CLIENT] Trying 0.in.pool.ntp.org
[NTP_CLIENT] Resolved -> 142.93.213.206
[NTP] Got time: 17:25:10 IST
[SPEECH] Announcing 17:25:10 IST
[SPEECH] Token stream begin
  > TOKEN THE
  > TOKEN TIME
  > TOKEN IS
  > TOKEN SEVENTEEN
  > TOKEN HOURS
  > TOKEN TWENTY FIVE
  > TOKEN MINUTES
  > TOKEN AND
  > TOKEN TEN
  > TOKEN SECONDS
  > END

Announcement #1 (17:25:10)
🔊 The time is seventeen hours twenty five minutes and ten seconds

[SPEECH] Token stream end

--- NTP Request ---
[KEY] Time request received
[NTP] Fetching time...
[NTP_CLIENT] Cooling down for 8895 ms
[NTP_CLIENT] Trying 0.in.pool.ntp.org
[NTP_CLIENT] Resolved -> 142.93.213.206
[NTP] Got time: 17:25:29 IST
[SPEECH] Announcing 17:25:29 IST
[SPEECH] Token stream begin
  > TOKEN THE
  > TOKEN TIME
  > TOKEN IS
  > TOKEN SEVENTEEN
  > TOKEN HOURS
  > TOKEN TWENTY FIVE
  > TOKEN MINUTES
  > TOKEN AND
  > TOKEN TWENTY NINE
  > TOKEN SECONDS
  > END

Announcement #2 (17:25:29)
🔊 The time is seventeen hours twenty five minutes and twenty nine seconds

[SPEECH] Token stream end

```

Figure 1 — QEMU console output showing boot sequence, DHCP, keypress, NTP resolution, IST conversion, TOKEN stream, and [BRIDGE] Done.

## 5.5 Known Limitations

- **Network:** UDP port 123 must be reachable from the host. A restrictive firewall will cause `ntp_get_time()` to fail after the 5-second receive timeout.
- **Precision:** Time resolution is one second. Sub-second precision is not required by the assignment.
- **TTS engine:** `tts_bridge.py` depends on a system TTS engine being installed: say on macOS (built-in), `espeak` or `espeak-ng` on Linux, or the PowerShell `SpeechSynthesizer` on Windows.
- **Queue depth:** `time_queue` has depth 1. If the user presses 't' while `speech_task` is still emitting tokens, the NTP result is discarded and the user must press 't' again.